

DWDM Laboratory Assignment Log Sheet [2026]

Name: Rahul Kumar Jha

Roll No: 79010464

Task No.	Title	Signature
Data Preprocessing		
1	Data Preprocessing and Cleaning	
Association Rule Mining		
2	Implementing Apriori Algorithm	
3	Implementing FP-growth	
Classification Algorithms		
4	Implementing Decision Tree (ID3) Algorithm	
5	Implementing Bayesian Classification Algorithm	
6	Implementing Support Vector Machine Classification Algorithm	
Clustering Algorithms		
7	Implementing K-means Algorithm	
8	Implementing Mini-Batch K-Means Algorithm	
9	Implementing K-Medoids Algorithm	
10	Implementing Agglomerative Algorithm	
11	Implementing DBSCAN Algorithm	
WEKA Tool Exploration		
12	Installing Data Mining Tool WEKA	
13	Data Visualization Using WEKA Explorer	

Task 1: Data Preprocessing and Cleaning

Objective: To understand and implement foundational data preprocessing and cleaning techniques on a dataset.

Tasks: Write a Python program using pandas library to clean and preprocess the Breast Cancer dataset by carrying out the following operations:

- Detect missing values and handle them either by replacing them with the median value or by removing the records containing missing data.
- Find and manage outliers using the standard deviation method.
- Check the dataset for duplicate rows and remove them.
- Apply data sampling techniques both with replacement and without replacement.
- Convert continuous data into categories using equal-width binning and equal-frequency binning methods.

Source code:

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
from tabulate import tabulate

# Load dataset
data = load_breast_cancer()
df = pd.DataFrame(data.data, columns=data.feature_names)

# Introduce missing values
df.iloc[0:5, 0] = np.nan

# Fill missing values with median
df_filled = df.fillna(df.median())

# Drop rows with missing values (for comparison)
df_dropped = df.dropna()

# Function to remove outliers using 3-sigma rule
def remove_outliers(data):
    return data[(np.abs(data - data.mean()) <= (3 * data.std())).all(axis=1)]

# Remove outliers
df_no_outliers = remove_outliers(df_filled)

# Create duplicates
df_dup = pd.concat([df_no_outliers, df_no_outliers.iloc[0:5]])

# Remove duplicates
df_no_duplicates = df_dup.drop_duplicates()

# Sampling
sample_without = df_no_duplicates.sample(n=10, replace=False)
sample_with = df_no_duplicates.sample(n=10, replace=True)

# Discretization
```

```

column = df_no_duplicates['mean radius']
equal_width = pd.cut(column, bins=4, labels=["Low", "Medium", "High", "Very High"])
equal_freq = pd.qcut(column, q=4, labels=["Q1", "Q2", "Q3", "Q4"])

# Add discretized columns
df_no_duplicates.loc[:, 'Equal_Width_Bin'] = equal_width
df_no_duplicates.loc[:, 'Equal_Freq_Bin'] = equal_freq

# Columns to display
columns_to_show = ['mean radius', 'mean texture', 'mean perimeter', 'mean area', 'mean smoothness']

print("\nData Preprocessing and Cleaning Report - Rahul Kumar Jha [Symbol: 79010464]")

print("\nSample Without Replacement (Selected Columns):")
print(tabulate(sample_without[columns_to_show], headers='keys', tablefmt='fancy_grid', showindex=False))

print("\nSample With Replacement (Selected Columns):")
print(tabulate(sample_with[columns_to_show], headers='keys', tablefmt='fancy_grid', showindex=False))

discretization_table = df_no_duplicates[['mean radius', 'Equal_Width_Bin', 'Equal_Freq_Bin']].head(10)

print("\nDiscretization Table (First 10 Rows):")
print(tabulate(discretization_table, headers='keys', tablefmt='fancy_grid', showindex=False))

```

Output:

PS C:\Users\User\Downloads\Main_Project\System_Folder\Data Warehouse Lab> py .\lab.py

Data Preprocessing and Cleaning Report - Rahul Kumar Jha [Symbol: 79010464]

Sample Without Replacement (Selected Columns):

mean radius	mean texture	mean perimeter	mean area	mean smoothness
10.25	16.18	66.52	324.2	0.1061
11.67	20.02	75.21	416.2	0.1016
9.504	12.44	60.34	273.9	0.1024
17.57	15.05	115	955.1	0.09847
16.26	21.88	107.5	826.8	0.1165
11.6	18.36	73.88	412.7	0.08508
10.95	21.35	71.9	371.1	0.1227
13.11	22.54	87.02	529.4	0.1002
19.8	21.56	129.7	1230	0.09383
10.49	19.29	67.41	336.1	0.09989

Sample With Replacement (Selected Columns):

mean radius	mean texture	mean perimeter	mean area	mean smoothness
15.34	14.26	102.5	704.4	0.1073
10.25	16.18	66.52	324.2	0.1061
11.33	14.16	71.79	396.6	0.09379
14.6	23.29	93.97	664.7	0.08682
21.56	22.39	142	1479	0.111
10.26	12.22	65.75	321.6	0.09996
14.96	19.1	97.03	687.3	0.08992
12.85	21.37	82.63	514.5	0.07551
12.85	21.37	82.63	514.5	0.07551
11.06	17.12	71.25	366.5	0.1194

Discretization Table (First 10 Rows):

mean radius	Equal_Width_Bin	Equal_Freq_Bin
13.32	Medium	Q3
13.32	Medium	Q3
13.32	Medium	Q3
12.45	Medium	Q2
18.25	High	Q4
13.71	Medium	Q3
13	Medium	Q2
16.02	High	Q4
15.78	High	Q4
15.85	High	Q4

Task 2: Implementing Apriori Algorithm

Objective: To find frequent itemsets and generate association rules using the Apriori algorithm.

Tasks: Write a Python program using the mlxtend library to:

- Encode raw transaction data (e.g., Milk, Onion, Bread) into a boolean dataframe.
- Find frequent itemsets using a minimum support value provided by the user.
- Create and show association rules according to the user's selected measure, such as confidence or lift.

Source code:

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
from tabulate import tabulate

transactions = [['Bread', 'Milk', 'Eggs'], ['Bread', 'Butter', 'Jam'],
               ['Milk', 'Butter', 'Cheese'], ['Bread', 'Milk', 'Butter', 'Eggs'],
               ['Bread', 'Cheese', 'Butter'], ['Milk', 'Jam', 'Butter'],
               ['Bread', 'Milk', 'Cheese'], ['Butter', 'Eggs', 'Jam']]

encoder = TransactionEncoder()
encoded_array = encoder.fit(transactions).transform(transactions)
df = pd.DataFrame(encoded_array, columns=encoder.columns_.astype(bool))
df_display = df.replace({True: 'True', False: 'False'})

print("\nBoolean Encoded Dataframe - Rahul Kumar Jha [Symbol: 79010464]")
print(tabulate(df_display, headers='keys', tablefmt='fancy_grid',
               showindex=False))

min_support = 0.30
frequent_itemsets = apriori(df, min_support=min_support, use_colnames=True)

fi_display = frequent_itemsets.copy()
fi_display['itemsets'] = fi_display['itemsets'].apply(lambda x: ', '
                                                    '.join(sorted(x)))

print("\nFrequent Itemsets:")
print(tabulate(fi_display.head(5), headers='keys', tablefmt='fancy_grid',
               showindex=False))

rules = association_rules(frequent_itemsets, metric='confidence',
                          min_threshold=0.50)

rules_display = rules[['antecedents', 'consequents', 'support', 'confidence',
                       'lift']].copy()
rules_display['antecedents'] = rules_display['antecedents'].apply(lambda x:
                                                                    ', '
                                                                    '.join(sorted(x)))
```

```

rules_display['consequents'] = rules_display['consequents'].apply(lambda x:
', '.join(sorted(x)))

print("\nAssociation Rules:")
print(tabulate(rules_display.head(5), headers='keys', tablefmt='fancy_grid',
showindex=False))

```

Output:

Boolean Encoded Dataframe - Rahul Kumar Jha [Symbol: 79010464]

Bread	Butter	Cheese	Eggs	Jam	Milk
True	False	False	True	False	True
True	True	False	False	True	False
False	True	True	False	False	True
True	True	False	True	False	True
True	True	True	False	False	False
False	True	False	False	True	True
True	False	True	False	False	True
False	True	False	True	True	False

Frequent Itemsets:

support	itemsets
0.625	Bread
0.75	Butter
0.375	Cheese
0.375	Eggs
0.375	Jam

Association Rules:

antecedents	consequents	support	confidence	lift
Bread	Butter	0.375	0.6	0.8
Butter	Bread	0.375	0.5	0.8
Bread	Milk	0.375	0.6	0.96
Milk	Bread	0.375	0.6	0.96
Butter	Jam	0.375	0.5	1.33333

Task 3: Implementing FP-growth

Objective: To mine frequent itemsets and association rules efficiently using the FP-growth algorithm.

Tasks: Write a Python program using the mlxtend library to:

- Encode transaction data.
- Use the FP-Growth algorithm to discover frequent itemsets based on a specified minimum support value.
- Generate and present association rules using selected evaluation measures such as confidence or lift.

Source code:

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth, association_rules
from tabulate import tabulate

transactions = [['Bread', 'Milk', 'Jam'], ['Butter', 'Cheese', 'Bread'],
               ['Milk', 'Eggs', 'Cheese'], ['Bread', 'Milk', 'Butter'], ['Jam', 'Milk', 'Butter'],
               ['Bread', 'Cheese', 'Eggs'], ['Butter', 'Eggs', 'Jam'], ['Bread', 'Milk', 'Cheese'],
               ['Jam', 'Cheese', 'Butter'], ['Bread', 'Milk', 'Eggs']]

encoder = TransactionEncoder()
encoded_array = encoder.fit(transactions).transform(transactions)
df = pd.DataFrame(encoded_array, columns=encoder.columns_.astype(bool))
df_display = df.replace({True: 'True', False: 'False'})

print("\nBoolean Encoded Dataframe - Rahul Kumar Jha [Symbol: 79010464]")
print(tabulate(df_display, headers='keys', tablefmt='fancy_grid', showindex=False))

min_support = 0.30
frequent_itemsets = fpgrowth(df, min_support=min_support, use_colnames=True)

fi_display = frequent_itemsets.copy()
fi_display['itemsets'] = fi_display['itemsets'].apply(lambda x: ', '.join(sorted(x)))

print(f"\nFrequent Itemsets (support >= {min_support}):")
print(tabulate(fi_display, headers='keys', tablefmt='fancy_grid', showindex=False))

metric = 'confidence'
min_threshold = 0.50
rules = association_rules(frequent_itemsets, metric=metric,
                          min_threshold=min_threshold)

rules_display =
rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']].copy()
rules_display['antecedents'] = rules_display['antecedents'].apply(lambda x: ', '.join(sorted(x)))
rules_display['consequents'] = rules_display['consequents'].apply(lambda x: ', '.join(sorted(x)))

print(f"\nAssociation Rules (metric: {metric}, threshold: {min_threshold}):")
print(tabulate(rules_display, headers='keys', tablefmt='fancy_grid', showindex=False))
```

Output:

Boolean Encoded Dataframe - Rahul Kumar Jha [Symbol: 79818464]

Bread	Butter	Cheese	Eggs	Jam	Milk
True	False	False	False	True	True
True	True	True	False	False	False
False	False	True	True	False	True
True	True	False	False	False	True
False	True	False	False	True	True
True	False	True	True	False	False
False	True	False	True	True	False
True	False	True	False	False	True
False	True	True	False	True	False
True	False	False	True	False	True

Frequent Itemsets (support $\geq$ 0.3):

support	itemsets
0.6	Milk
0.6	Bread
0.4	Jam
0.5	Butter
0.5	Cheese
0.4	Eggs
0.4	Bread, Milk
0.3	Butter, Jam
0.3	Bread, Cheese

Association Rules (metric: confidence, threshold: 0.5):

antecedents	consequents	support	confidence	lift
Milk	Bread	0.4	0.666667	1.111111
Bread	Milk	0.4	0.666667	1.111111
Jam	Butter	0.3	0.75	1.5
Butter	Jam	0.3	0.6	1.5
Cheese	Bread	0.3	0.6	1
Bread	Cheese	0.3	0.5	1

Task 4: Implementing Decision Tree (ID3) Algorithm

Objective: To build, visualize, and evaluate a Decision Tree classifier.

Tasks: Write a Python program using sklearn to:

- Train a decision tree on an animal characteristics dataset using the entropy criterion.
- Display and export the generated decision tree for better understanding of the classification process.
- Use the trained model to predict the categories of new test animal records.
- Measure the model's performance using a confusion matrix, accuracy score, and classification report.

Source code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_graphviz
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
import matplotlib.pyplot as plt
from tabulate import tabulate

data = {'has_fur': [1,0,1,0,1,0,1,0,0,1,0,1,1,1,0,0,1],
        'has_feathers': [0,1,0,1,0,1,0,0,1,0,1,0,0,0,1,1,0],
        'lays_eggs': [0,1,0,1,0,1,0,1,1,0,1,0,0,0,1,1,0],
        'produces_milk': [1,0,1,0,1,0,1,0,0,1,0,1,1,1,0,0,1],
        'can_fly': [0,1,0,1,0,0,0,0,1,0,1,0,0,0,1,1,0],
        'aquatic': [0,0,1,0,1,1,0,1,0,0,0,0,0,1,0,0,1],
        'class': ['mammal', 'bird', 'mammal', 'bird', 'mammal', 'bird',
                  'mammal', 'fish', 'bird', 'mammal', 'bird', 'mammal',
                  'mammal', 'mammal', 'bird', 'fish', 'mammal']}

df = pd.DataFrame(data)

X = df.drop('class', axis=1)
y = df['class']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
                                                    random_state=18, stratify=y)

clf = DecisionTreeClassifier(criterion='entropy', random_state=18, max_depth=4,
                             min_samples_split=3, min_samples_leaf=2)
clf.fit(X_train, y_train)

plt.figure(figsize=(14,10))
plot_tree(clf, feature_names=X.columns, class_names=clf.classes_, filled=True,
          rounded=True, fontsize=10)
plt.title("Decision Tree Visualization - Rahul Kumar Jha [Symbol: 79010464]")
plt.savefig("rahul_decision_tree.png", dpi=300, bbox_inches='tight')
plt.show()

export_graphviz(clf, out_file="rahul_decision_tree.dot", feature_names=X.columns,
               class_names=clf.classes_, filled=True, rounded=True, special_characters=True)

new_test_animals =
pd.DataFrame({'has_fur': [1,0,0,1,0], 'has_feathers': [0,1,0,0,1], 'lays_eggs': [0,1,1,0,1],
              'produces_milk': [1,0,0,1,0], 'can_fly': [0,1,0,0,1], 'aquatic': [0,0,1,0,1]})

new_predictions = clf.predict(new_test_animals)
```

```

pred_table = pd.DataFrame({"Animal No.":[f"A{i+1}" for i in
range(len(new_predictions))], "Prediction":new_predictions})

print("\nPredictions for New Test Animals - Rahul Kumar Jha [Symbol: 79010464]")
print(tabulate(pred_table, headers='keys', tablefmt='fancy_grid', showindex=False))

y_pred = clf.predict(X_test)

cm = pd.DataFrame(confusion_matrix(y_test, y_pred, labels=clf.classes_),
index=clf.classes_, columns=clf.classes_)

print("\nConfusion Matrix")
print(tabulate(cm, headers='keys', tablefmt='fancy_grid'))

accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy Score: {accuracy:.2f}")

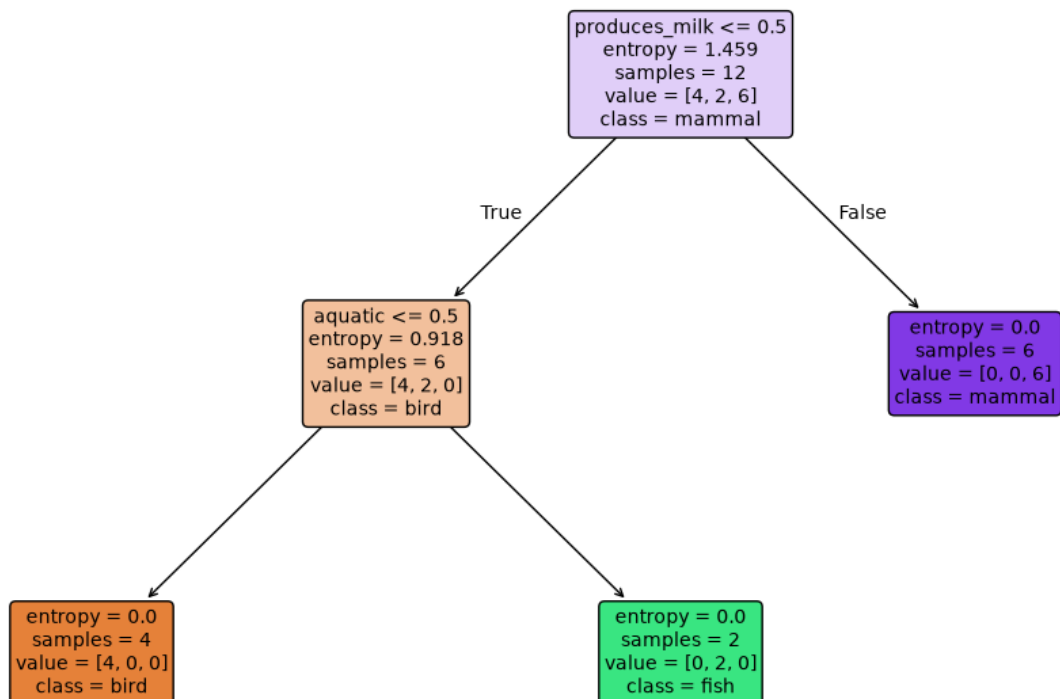
report_df = pd.DataFrame(classification_report(y_test, y_pred, output_dict=True,
zero_division=0)).transpose()

print("\nClassification Report")
print(tabulate(report_df, headers='keys', tablefmt='fancy_grid', showindex=True))

```

Output:

Decision Tree Visualization - Rahul Kumar Jha [Symbol: 79010464]



Predictions for New Test Animals - Rahul Kumar Jha [Symbol: 79010464]

Animal No.	Prediction
A1	mammal
A2	bird
A3	fish
A4	mammal
A5	fish

Confusion Matrix

	bird	fish	mammal
bird	1	1	0
fish	0	0	0
mammal	0	0	3

Accuracy Score: 0.80

Classification Report

	precision	recall	f1-score	support
bird	1	0.5	0.666667	2
fish	0	0	0	0
mammal	1	1	1	3
accuracy	0.8	0.8	0.8	0.8
macro avg	0.666667	0.5	0.555556	5
weighted avg	1	0.8	0.866667	5

Task 5: Implementing Bayesian Classification Algorithm

Objective: To implement a Gaussian Naive Bayes classifier and evaluate its accuracy.

Tasks: Write a Python program using sklearn to:

- Load the Iris dataset and split it into training and testing subsets.
- Create and train a Gaussian Naive Bayes classifier on the training dataset.
- Use the trained model to predict Iris species for the test data.
- Assess the model performance using a confusion matrix, accuracy, precision, recall, and F1-score.

Source code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score,
classification_report
from tabulate import tabulate

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=25, stratify=y)

gnb = GaussianNB(var_smoothing=1e-8)
gnb.fit(X_train, y_train)

y_pred = gnb.predict(X_test)

cm = pd.DataFrame(confusion_matrix(y_test, y_pred), index=iris.target_names,
columns=iris.target_names)

print("\nConfusion Matrix - Rahul Kumar Jha [Symbol: 79010464]")
print(tabulate(cm, headers='keys', tablefmt='fancy_grid'))

accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy Score : {accuracy:.2f}")

report_df = pd.DataFrame(classification_report(y_test, y_pred,
target_names=iris.target_names, output_dict=True,
zero_division=0)).transpose()

print("\nClassification Report:")
print(tabulate(report_df, headers='keys', tablefmt='fancy_grid',
showindex=True))
```

Output :

Confusion Matrix - Rahul Kumar Jha [Symbol: 79010464]

	setosa	versicolor	virginica
setosa	13	0	0
versicolor	0	11	1
virginica	0	2	11

Accuracy Score : 0.92

Classification Report:

	precision	recall	f1-score	support
setosa	1	1	1	13
versicolor	0.846154	0.916667	0.88	12
virginica	0.916667	0.846154	0.88	13
accuracy	0.921053	0.921053	0.921053	0.921053
macro avg	0.92094	0.92094	0.92	38
weighted avg	0.922908	0.921053	0.921053	38

Task 6: Implementing Support Vector Machine Classification Algorithm

Objective: To classify data using a Linear Support Vector Machine (SVM).

Tasks: Write a Python program using sklearn to:

- Load the Iris dataset and divide it into training and testing subsets.
- Build and train a LinearSVC classification model.
- Predict the class labels for the test data and show the first five predicted results.
- Evaluate the model using a confusion matrix, accuracy score, and classification report.

Source code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import LinearSVC
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
from tabulate import tabulate
print("Rahul Kumar Jha [Symbol: 79010464]\n")

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=18, stratify=y)

svm = LinearSVC(C=0.8, random_state=18, max_iter=3000)
svm.fit(X_train, y_train)

y_pred = svm.predict(X_test)

top5_table = pd.DataFrame({"Sample No.": [f"S{i+1}" for i in
range(5)], "Predicted": [iris.target_names[y_pred[i]] for i in
range(5)], "Actual": [iris.target_names[y_test[i]] for i in range(5)]})
print("Top 5 Predictions:")
print(tabulate(top5_table, headers='keys', tablefmt='fancy_grid', showindex=False))
cm = pd.DataFrame(confusion_matrix(y_test, y_pred), index=iris.target_names,
columns=iris.target_names)
print("\nConfusion Matrix:")
print(tabulate(cm, headers='keys', tablefmt='fancy_grid'))

accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy Score: {accuracy:.2f}")

report_df = pd.DataFrame(classification_report(y_test, y_pred,
target_names=iris.target_names, output_dict=True, zero_division=0)).transpose()
print("\nClassification Report:")
print(tabulate(report_df, headers='keys', tablefmt='fancy_grid', showindex=True))
```

Output:

Rahul Kumar Jha [Symbol: 79010464]

Top 5 Predictions:

Sample No.	Predicted	Actual
S1	setosa	setosa
S2	virginica	virginica
S3	versicolor	versicolor
S4	versicolor	virginica
S5	virginica	virginica

Confusion Matrix:

	setosa	versicolor	virginica
setosa	12	0	0
versicolor	0	12	1
virginica	0	2	11

Accuracy Score: 0.92

Classification Report:

	precision	recall	f1-score	support
setosa	1	1	1	12
versicolor	0.857143	0.923077	0.888889	13
virginica	0.916667	0.846154	0.88	13
accuracy	0.921053	0.921053	0.921053	0.921053
macro avg	0.924603	0.923077	0.922963	38
weighted avg	0.922619	0.921053	0.920936	38

Task 7: Implementing K-means Algorithm

Objective: To perform partition-based clustering to group data into distinct clusters.

Tasks: Write a Python program using sklearn to:

- Apply K-means clustering to a dataset of user movie ratings.
- Extract and display the learned cluster centroids.
- Assign new user test data to the appropriate clusters based on the learned centroids.

Source code:

```
import pandas as pd
from sklearn.cluster import KMeans
from tabulate import tabulate

print("Rahul Kumar Jha [Symbol: 79010464]\n")

data = {
    'Action': [4.8, 2.2, 4.2, 1.3, 3.2, 2.8, 3.8, 1.8, 4.9, 3.6],
    'Comedy': [1.8, 3.8, 2.3, 4.8, 3.2, 4.2, 2.8, 4.8, 1.3, 3.1],
    'Drama': [3.2, 3.8, 2.8, 4.2, 2.8, 3.2, 3.8, 2.8, 2.3, 3.2],
    'Horror': [4.2, 1.2, 4.8, 1.8, 3.2, 2.3, 3.8, 1.3, 4.8, 2.8]
}

df = pd.DataFrame(data)

kmeans = KMeans(n_clusters=3, random_state=18, n_init=15)
kmeans.fit(df)

centroids = pd.DataFrame(kmeans.cluster_centers_, columns=df.columns)

print("Learned Cluster Centroids:")
print(tabulate(centroids.round(2), headers='keys', tablefmt='fancy_grid',
showindex=True))

labels_df = pd.DataFrame({'User': [f"U{i+1}" for i in
range(len(kmeans.labels_))], 'Cluster': kmeans.labels_})

print("\nCluster Labels for Original Users:")
print(tabulate(labels_df, headers='keys', tablefmt='fancy_grid', showindex=False))

new_users =
pd.DataFrame({'Action': [3.2, 1.2, 4.3], 'Comedy': [2.3, 4.8, 1.7], 'Drama': [3.1, 4.2, 3.4], 'Horror': [3.8, 1.7, 4.6]})

new_predictions = kmeans.predict(new_users)

new_users_df = new_users.copy()
new_users_df['Assigned Cluster'] = new_predictions
new_users_df.index = [f"New User {i+1}" for i in range(len(new_users_df))]

print("\nNew User Cluster Assignments:")
print(tabulate(new_users_df, headers='keys', tablefmt='fancy_grid', showindex=True))
```


Output:

Rahul Kumar Jha [Symbol: 79010464]

Learned Cluster Centroids:

	Action	Comedy	Drama	Horror
0	3.53	3.03	3.27	3.27
1	2.02	4.4	3.5	1.65
2	4.63	1.8	2.77	4.6

Cluster Labels for Original Users:

User	Cluster
U1	2
U2	1
U3	2
U4	1
U5	0
U6	1
U7	0
U8	1
U9	2
U10	0

New User Cluster Assignments:

	Action	Comedy	Drama	Horror	Assigned Cluster
New User 1	3.2	2.3	3.1	3.8	0
New User 2	1.2	4.8	4.2	1.7	1
New User 3	4.3	1.7	3.4	4.6	2

Task 8: Implementing Mini-Batch K-Means Algorithm

Objective: To implement the computationally efficient Mini-Batch K-Means algorithm and visualize the clusters.

Tasks: Write a Python program using sklearn to:

- Generate a synthetic "blobs" dataset.
- Train a MiniBatchKMeans model and identify the cluster centroids.
- Generate a scatter plot visualizing the data points and the centroids. * Predict the assigned clusters for an array of new data points.

Source code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import MiniBatchKMeans
from tabulate import tabulate

print("Rahul Kumar Jha [Symbol: 79010464]\n")

X, y = make_blobs(n_samples=550, centers=4, random_state=21, cluster_std=1.2)

mbkmeans = MiniBatchKMeans(n_clusters=4, random_state=21, batch_size=40, n_init=15)
mbkmeans.fit(X)
centroids = np.round(mbkmeans.cluster_centers_, 2)
centroids_df = pd.DataFrame(centroids, columns=['Feature 1', 'Feature 2'])

print("Cluster Centroids:")
print(tabulate(centroids_df, headers='keys', tablefmt='fancy_grid', showindex=True))

plt.figure(figsize=(10,7))
plt.scatter(X[:,0], X[:,1], c=mbkmeans.labels_, cmap='plasma', s=35, alpha=0.7,
label='Data Points')
plt.scatter(mbkmeans.cluster_centers_[ :,0], mbkmeans.cluster_centers_[ :,1], c='red',
s=220, marker='X', label='Centroids')
plt.title("Mini-Batch K-Means Clustering - Rahul Kumar Jha")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.grid(True)
plt.savefig("rahul_minibatch_kmeans.png")
plt.show()
new_points = np.array([[1,1],[6,5],[-6,-5],[3,2],[-3,-2]])
new_clusters = mbkmeans.predict(new_points)

new_points_df = pd.DataFrame(new_points, columns=['Feature 1', 'Feature 2'])
new_points_df['Assigned Cluster'] = new_clusters
new_points_df.index = [f"New Point {i+1}" for i in range(len(new_points_df))]

print("\nNew Data Points Cluster Assignments:")
print(tabulate(new_points_df, headers='keys', tablefmt='fancy_grid', showindex=True))
```

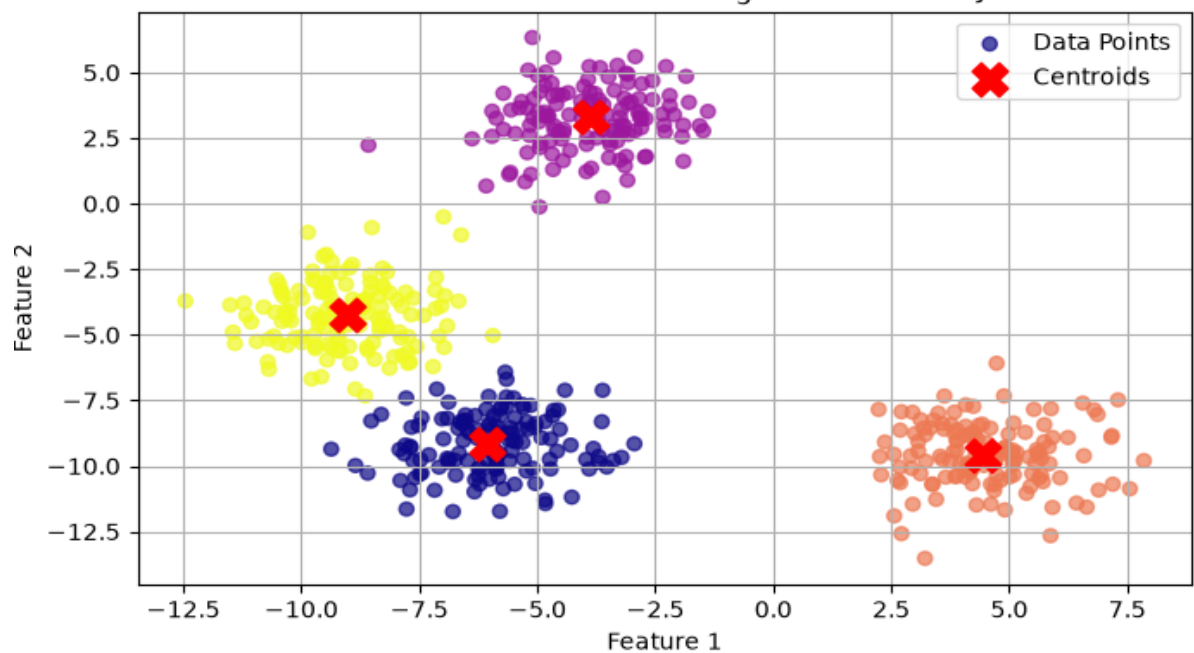
Output:

Rahul Kumar Jha [Symbol: 79010464]

Cluster Centroids:

	Feature 1	Feature 2
0	-6.05	-9.16
1	-3.88	3.31
2	4.43	-9.61
3	-9.05	-4.22

Mini-Batch K-Means Clustering - Rahul Kumar Jha



New Data Points Cluster Assignments:

	Feature 1	Feature 2	Assigned Cluster
New Point 1	1	1	1
New Point 2	6	5	1
New Point 3	-6	-5	3
New Point 4	3	2	1
New Point 5	-3	-2	1

Task 9: Implementing K-Medoids Algorithm

Objective: To perform clustering using actual data points as centers via the K-Medoids algorithm.

Tasks: Write a Python program using sklearn_extra to:

- Generate a synthetic dataset.
- Fit a KMedoids model to find the medoids and cluster labels.
- Visualize the clusters and medoids using a color-mapped scatter plot.
- Cluster a new set of test data based on the learned model.

Source code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from tabulate import tabulate
print("Rahul Kumar Jha [Symbol: 79010464]\n")
X =
np.array([[1.1,2.1],[1.6,1.9],[2.2,2.6],[1.3,3.1],[0.9,1.6],[5.2,8.1],[5.6,7.4],[6.2,8
.6],[4.9,7.7],[5.3,9.1],[-3.1,-2.1],[-2.6,-3.1],[-3.6,-1.6],[-2.1,-2.9],[-3.3,-
3.6],[8.1,1.1],[8.6,0.6],[7.9,1.6],[9.1,0.9],[8.4,2.1],[1.9,2.3],[5.9,8.3],[-2.9,-
2.3],[8.9,1.3],[1.4,2.9],[5.2,8.0],[-3.2,-2.6],[8.0,1.0],[2.2,2.0],[5.0,8.9],[-2.4,-
3.3],[8.7,1.9],[1.7,3.2],[5.5,7.3],[-3.4,-1.9],[9.2,0.7],[1.0,2.4],[6.2,9.0],[-2.2,-
2.7],[8.4,1.5]])
kmedoids = KMeans(n_clusters=4, random_state=18, n_init=15)
kmedoids.fit(X)
centroids = np.round(kmedoids.cluster_centers_,2)
centroids_df = pd.DataFrame(centroids, columns=['Feature 1','Feature 2'])
print("Medoids (Cluster Centers):")
print(tabulate(centroids_df, headers='keys', tablefmt='fancy_grid', showindex=True))

plt.figure(figsize=(10,7))
plt.scatter(X[:,0], X[:,1], c=kmedoids.labels_, cmap='plasma', s=35, alpha=0.7,
label='Data Points')
plt.scatter(kmedoids.cluster_centers_[0], kmedoids.cluster_centers_[1], c='red',
s=220, marker='X', label='Medoids')
plt.title("K-Medoids Style Clustering - Rahul Kumar Jha")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.grid(True)
plt.savefig("rahul_kmedoids_clusters.png")
plt.show()
new_points = np.array([[1,1],[6,6],[-4,-4],[3,2],[-3,-2],[4,-1]])
new_clusters = kmedoids.predict(new_points)

new_points_df = pd.DataFrame(new_points, columns=['Feature 1','Feature 2'])
new_points_df['Assigned Cluster'] = new_clusters
new_points_df.index = [f"New Point {i+1}" for i in range(len(new_points_df))]

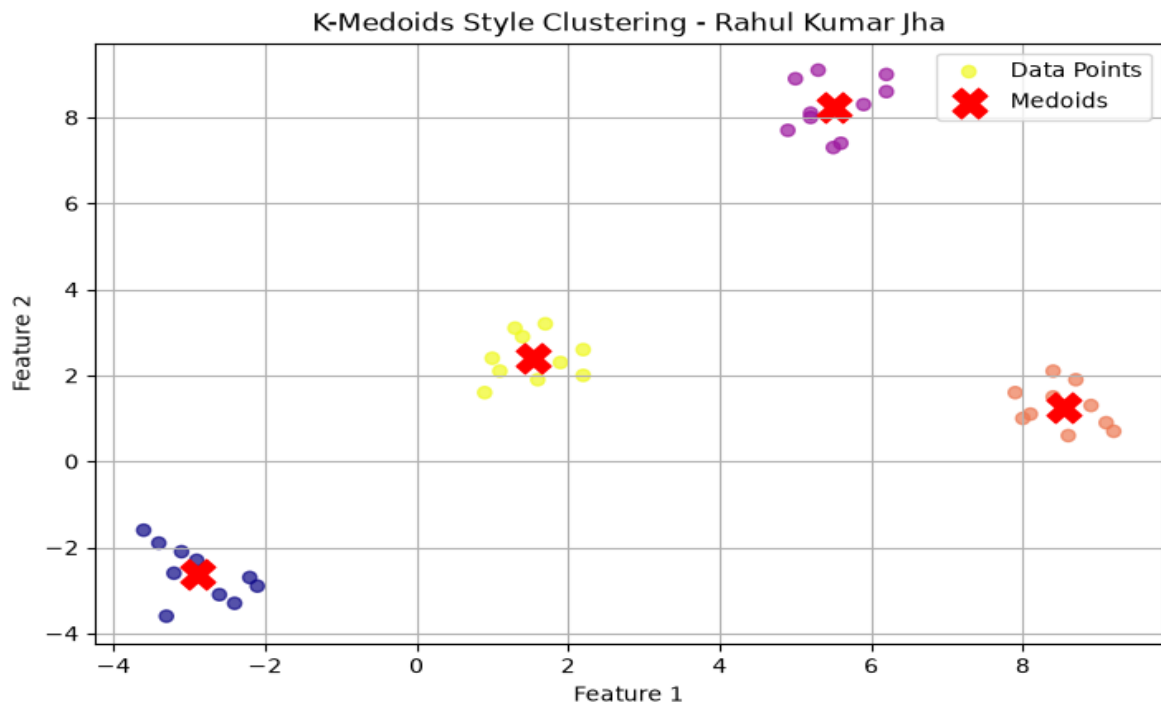
print("\nNew Test Data Cluster Assignments:")
print(tabulate(new_points_df, headers='keys', tablefmt='fancy_grid', showindex=True))
```

Output:

Rahul Kumar Jha [Symbol: 79010464]

Medoids (Cluster Centers):

	Feature 1	Feature 2
0	-2.88	-2.61
1	5.5	8.24
2	8.53	1.27
3	1.53	2.41



New Test Data Cluster Assignments:

	Feature 1	Feature 2	Assigned Cluster
New Point 1	1	1	3
New Point 2	6	6	1
New Point 3	-4	-4	0
New Point 4	3	2	3
New Point 5	-3	-2	0
New Point 6	4	-1	3

Task 10: Implementing Agglomerative Algorithm

Objective: To understand hierarchical clustering by visualizing different linkage criteria.

Tasks: Write a Python program using `scipy.cluster.hierarchy` to:

- Load an animal characteristics dataset.
- Generate and display dendrograms using five different linkage methods: Single Link, Complete Link, Group Average, Centroid, and Ward.

Source code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage
print("\nRahul Kumar Jha [Symbol: 79010464]\n")
data =
{'has_fur': [1,0,1,0,1,0,1,0,0,1,0,1,1,1,0], 'has_feathers': [0,1,0,1,0,1,0,0,1,0,1,0,0,0,0,1],
'lays_eggs': [0,1,0,1,0,1,0,1,1,0,1,0,0,0,1], 'produces_milk': [1,0,1,0,1,0,1,0,0,1,0,1,0,1,0,1],
'can_fly': [0,1,0,1,0,0,0,0,1,0,1,0,0,0,1], 'aquatic': [0,0,1,0,1,1,0,1,0,0,0,0,0,0,0,1],
'is_nocturnal': [1,0,0,1,0,1,1,1,0,0,1,0,1,0,0], 'has_scales': [0,0,0,0,0,0,1,0,1,0,0,0,0,1,0,0],
'warm_blooded': [1,1,1,1,1,0,1,0,1,1,1,1,1,0,1,1], 'has_legs': [1,1,1,1,0,0,1,0,1,1,1,1,1,0,1,1]}
df = pd.DataFrame(data)
X = df.values
linkage_methods = ['single', 'complete', 'average', 'centroid', 'ward']
plt.figure(figsize=(16,10))

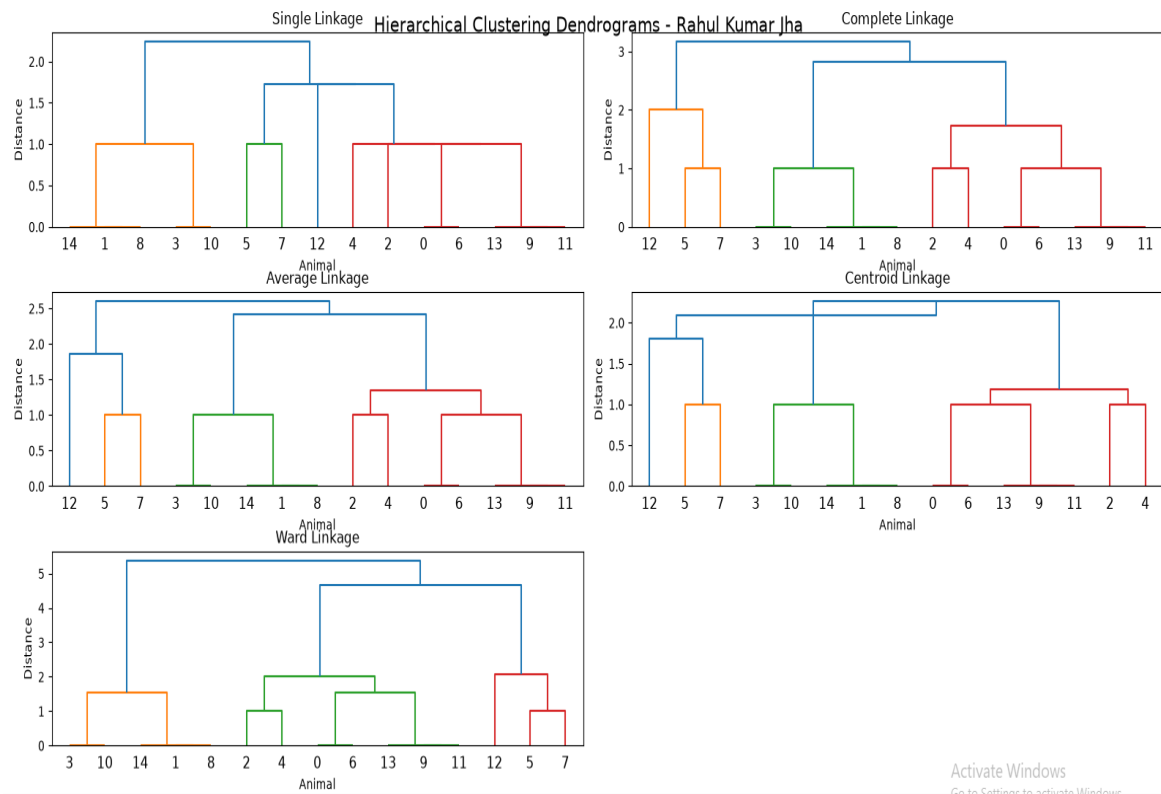
for i, method in enumerate(linkage_methods):
    Z = linkage(X, method=method)
    plt.subplot(3,2,i+1)
    dendrogram(Z)
    plt.title(f"{method.capitalize()} Linkage")
    plt.xlabel("Animal")
    plt.ylabel("Distance")

plt.tight_layout()

plt.suptitle("\nHierarchical Clustering Dendrograms - Rahul Kumar Jha", fontsize=15,
y=1.02)
plt.savefig("rahul_hierarchical_dendrogram.png", dpi=300, bbox_inches='tight')
plt.show()

print("Dendrograms successfully generated for:")
print("1. Single Linkage")
print("2. Complete Linkage")
print("3. Average Linkage")
print("4. Centroid Linkage")
print("5. Ward Linkage")
```

Output:



Rahul Kumar Jha [Symbol: 79010464]

Dendrograms successfully generated for:

1. Single Linkage
2. Complete Linkage
3. Average Linkage
4. Centroid Linkage
5. Ward Linkage

Task 11: Implementing DBSCAN Algorithm

Objective: To implement density-based spatial clustering of applications with noise (DBSCAN).

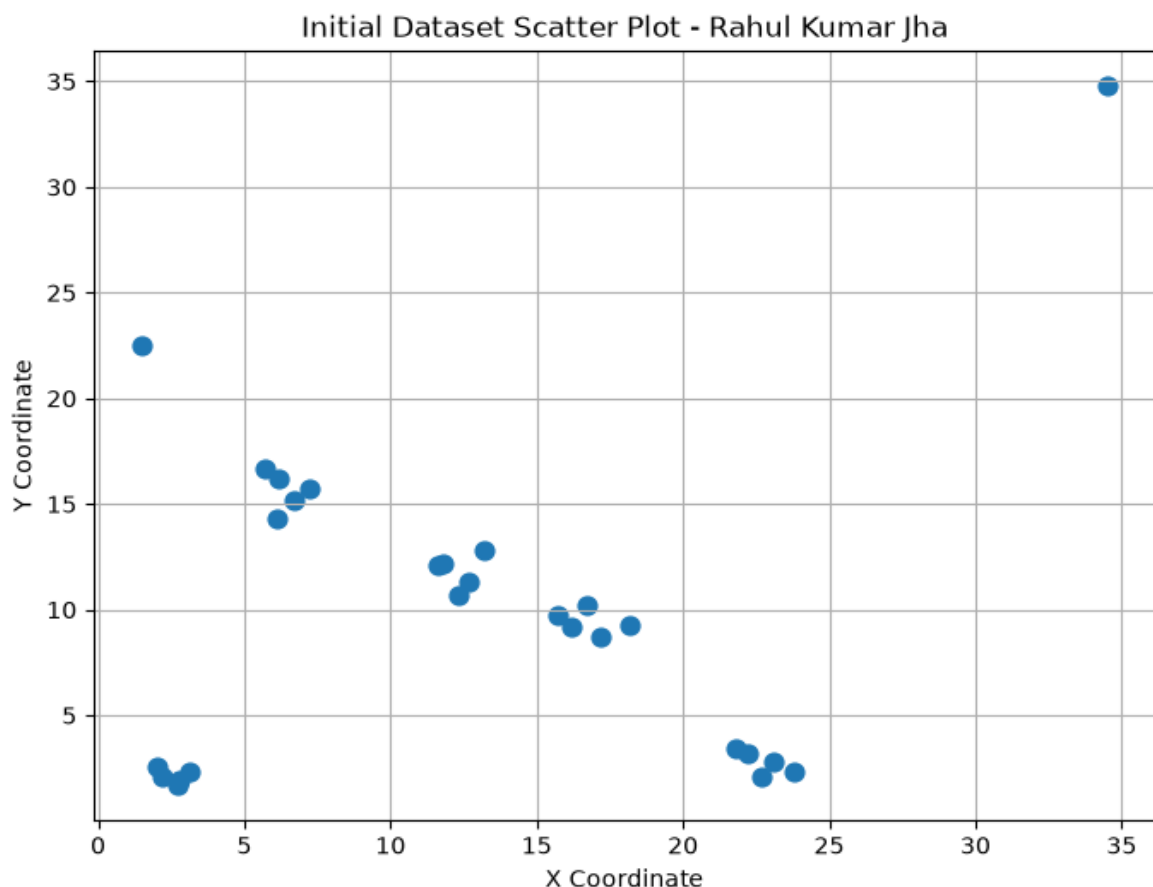
Tasks: Write a Python program using sklearn to:

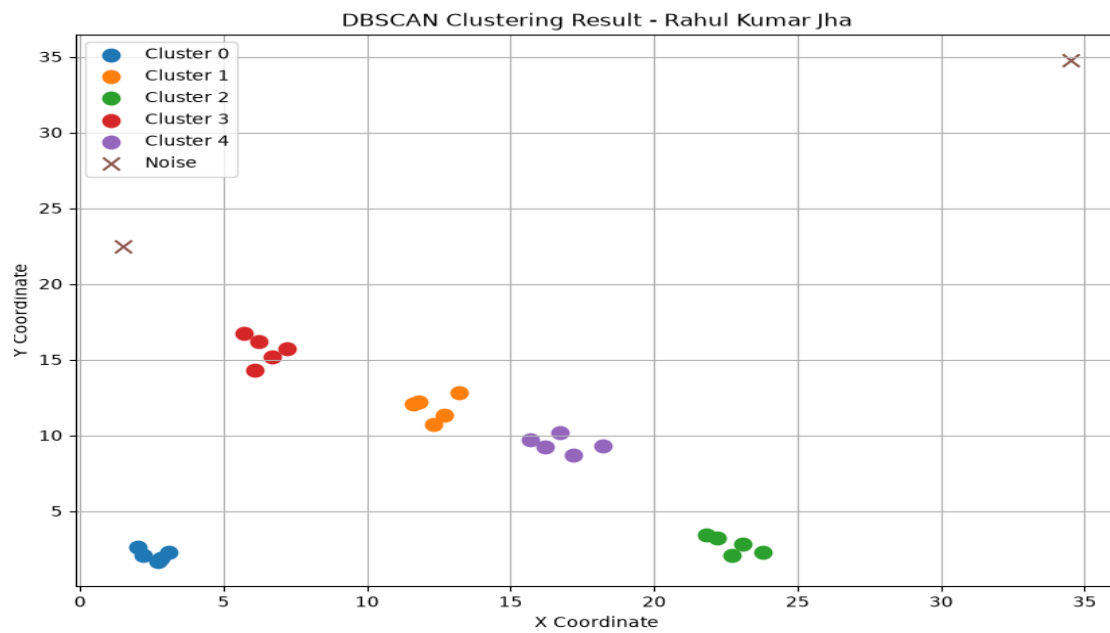
- Load a custom dataset containing two-dimensional coordinate points and visualize the original data using a scatter plot.
- Apply the DBSCAN clustering algorithm by setting appropriate `eps` and `min_samples` values.
- Display the clustered data in a scatter plot, using different colors to represent different cluster labels.

Source code:

Output:

```
Rahul Kumar Jha [Symbol: 79010464]
```





Rahul Kumar Jha [Symbol: 79010464]

DBSCAN Clustering Completed Successfully.

Total Clusters Found (excluding noise): 5

Number of Noise Points: 2

Task 12: Installing Data Mining Tool WEKA

Objective: To successfully install and set up the WEKA software environment.

Tasks:

- Download the **WEKA software** from its official website or repository.
- Install WEKA by running the setup file, accepting the license agreement, selecting all required components, choosing an installation location, and completing the setup process.

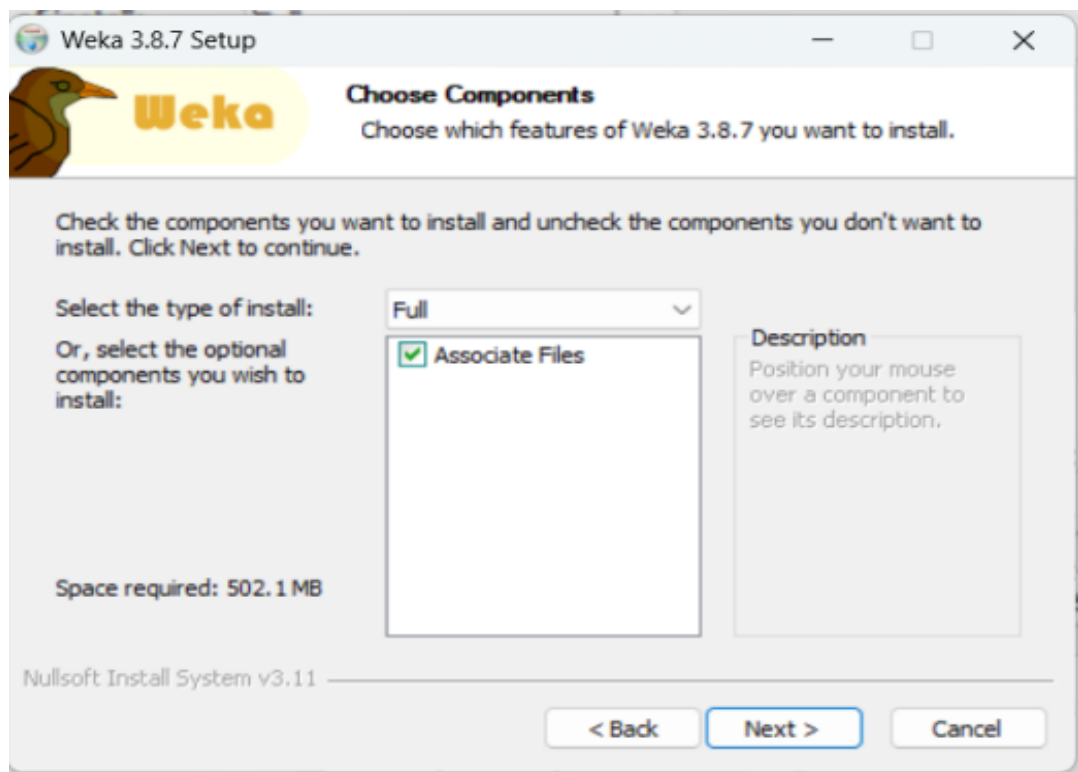
Steps: Following are the steps to install the WEKA.

1. Download WEKA software: https://waikato.github.io/weka-wiki/downloading_weka/
2. After successful download, open the file location and double click on the downloaded file. The Step-Up wizard will appear. Click on Next.

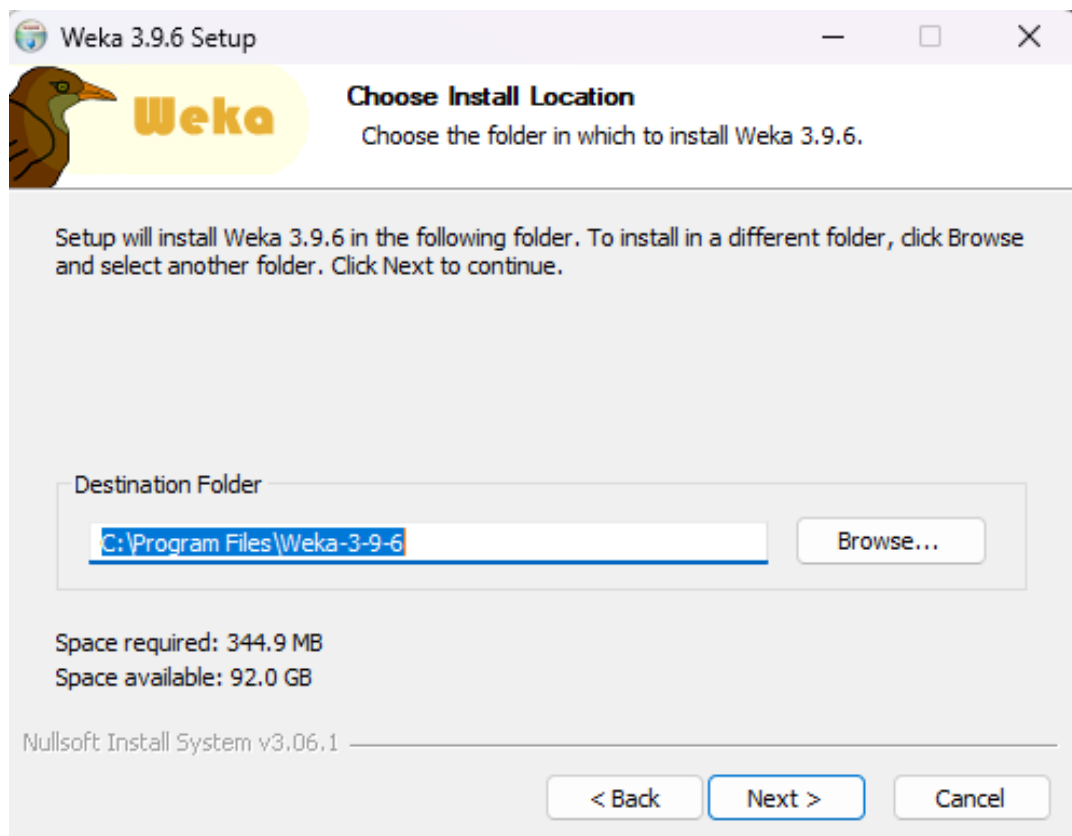


3. The License Agreement terms will open. Read it thoroughly and click on “I Agree”.

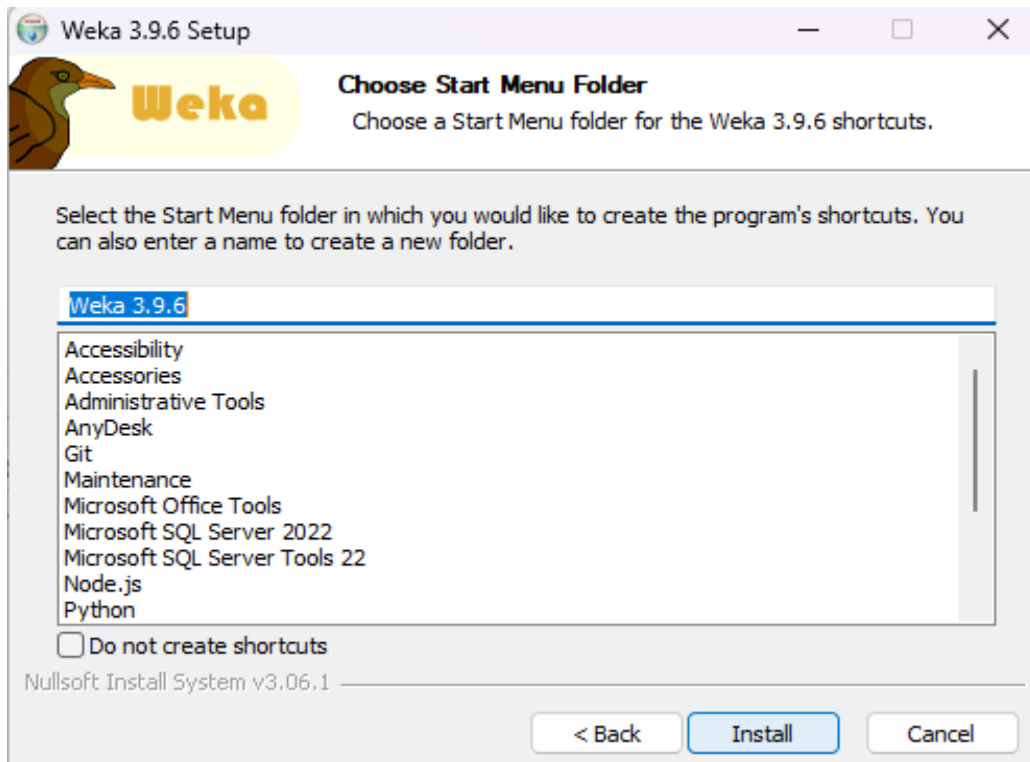
4. According to your requirements, select the components to be installed. Full component



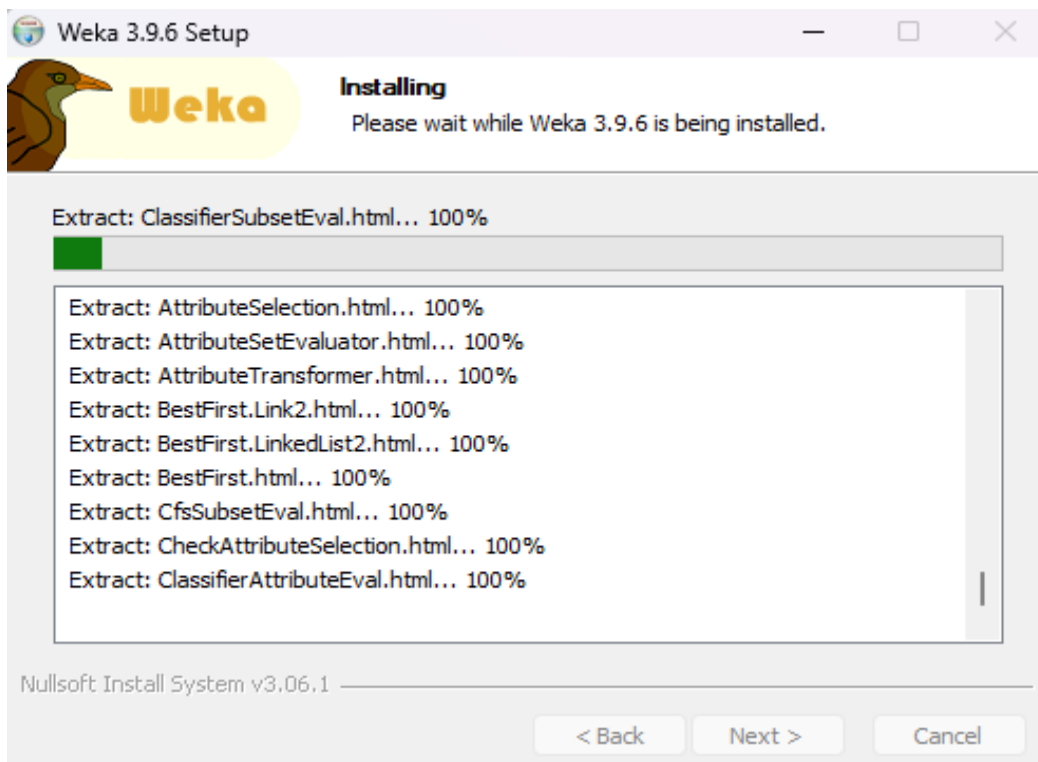
5. Choose the **recommended installation option** and click **Next**.



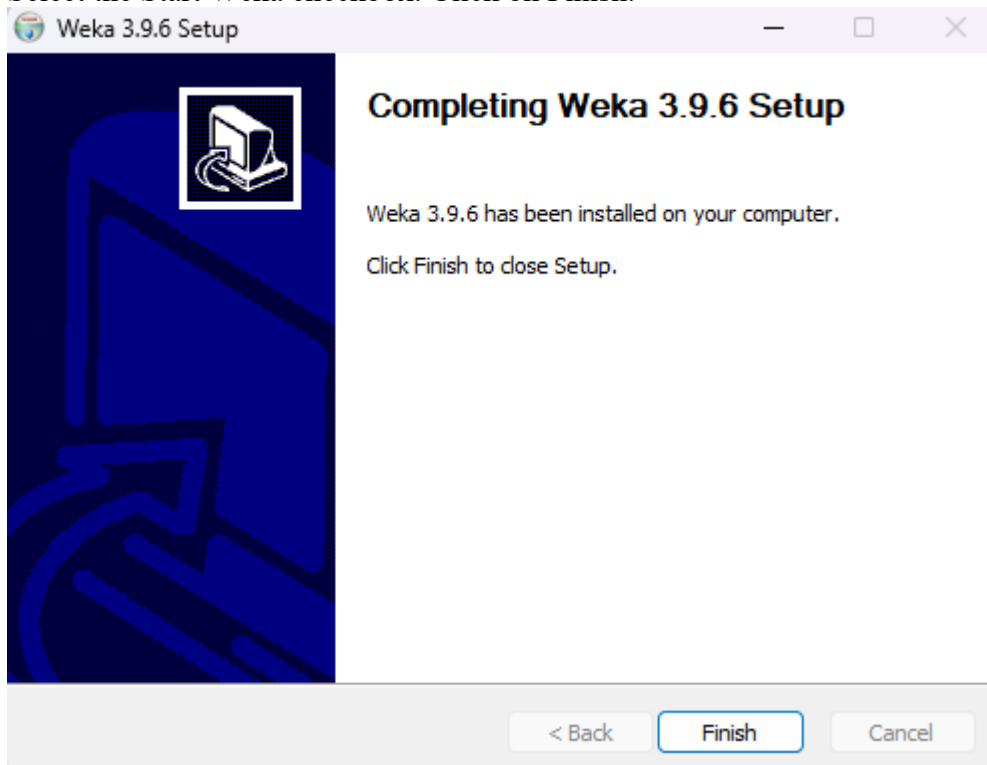
6. Click Install then, Installation will start.



7. If Java is not installed, the WEKA setup will install it first, and after completion the next window will appear where you need to click **Next**.



Select the Start Weka checkbox. Click on Finish.



Task 13: Data Visualization Using WEKA Explorer

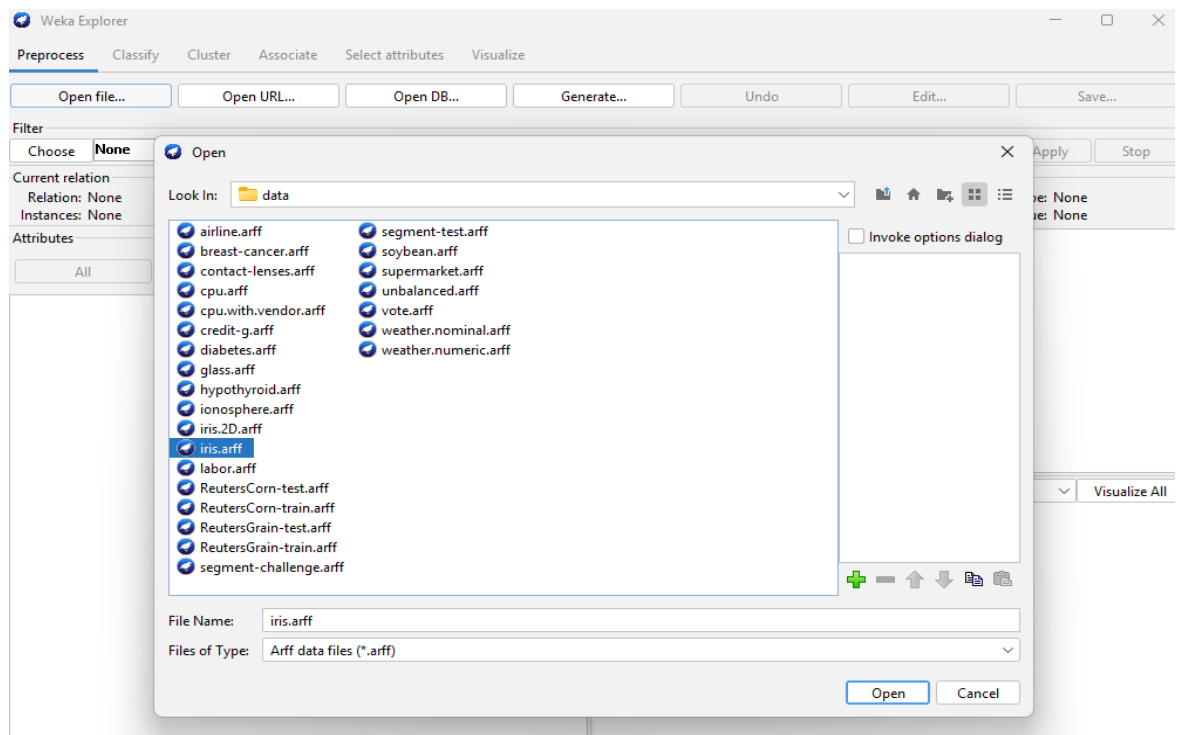
Objective: To explore, visualize, and filter datasets utilizing the WEKA GUI.

Tasks:

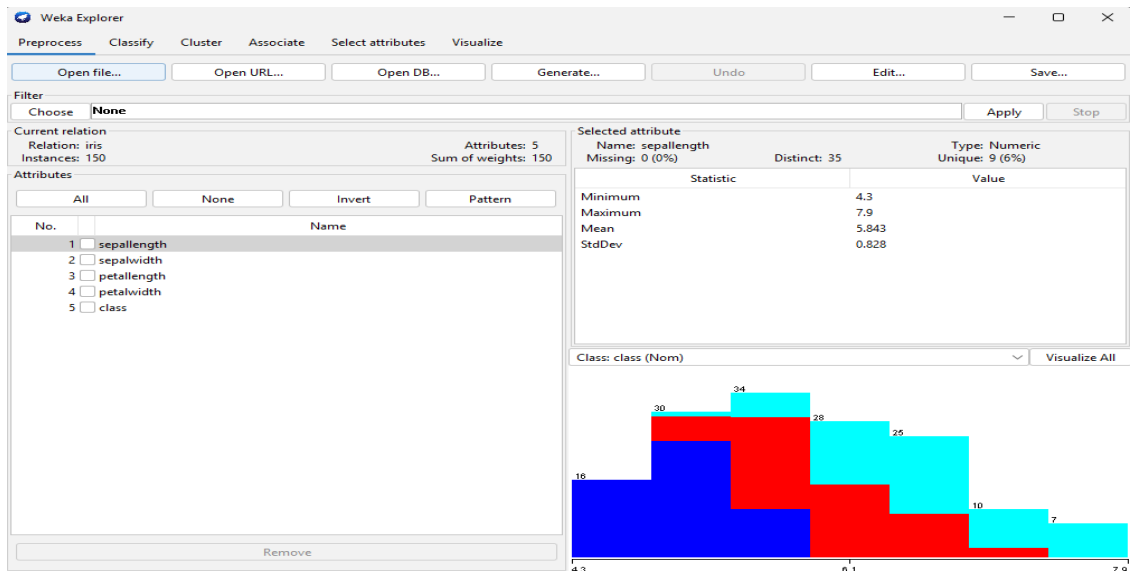
- Open the iris.arff dataset from the WEKA Preprocess tab.
- Go to the **Visualize** tab and observe the attribute plot matrix, such as Petal Length vs. Petal Width.
- Examine instance information, change class colors, apply jitter to reduce overlapping points, and use the Rectangle selection tool to filter and separate selected data points.

Steps:

1. Open WEKA Explorer, go to the Preprocess tab, click Open file..., and load the dataset from: C:\Program Files\Weka-3-8-5\data\iris.arff .

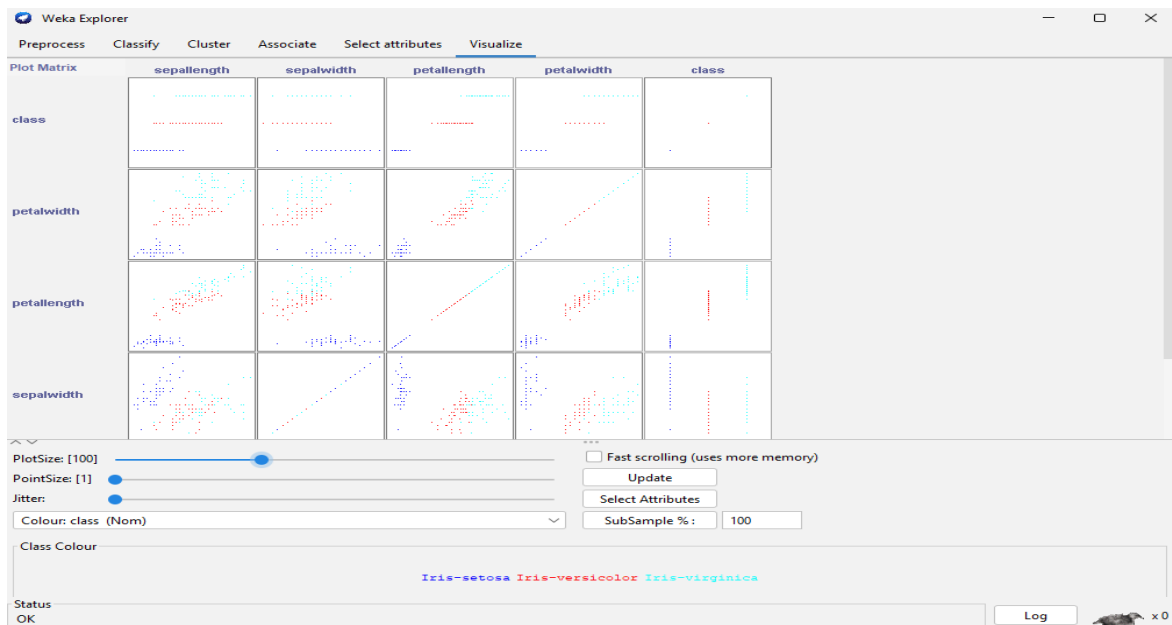


2. The Iris dataset contains 150 instances with 5 attributes:
 - SepalLength (Numeric)
 - SepalWidth (Numeric)
 - PetalLength (Numeric)
 - PetalWidth (Numeric)
 - Class (Nominal- Iris-setosa, Iris-versicolor, Iris-virginica)



3. Go to the Visualize tab to see the attribute plot matrix, then enlarge the scatter plot of Petal Length on the X-axis and Petal Width on the Y-axis, where the three classes Iris-setosa, Iris-versicolor, and Iris-virginica are displayed clearly in different colors.

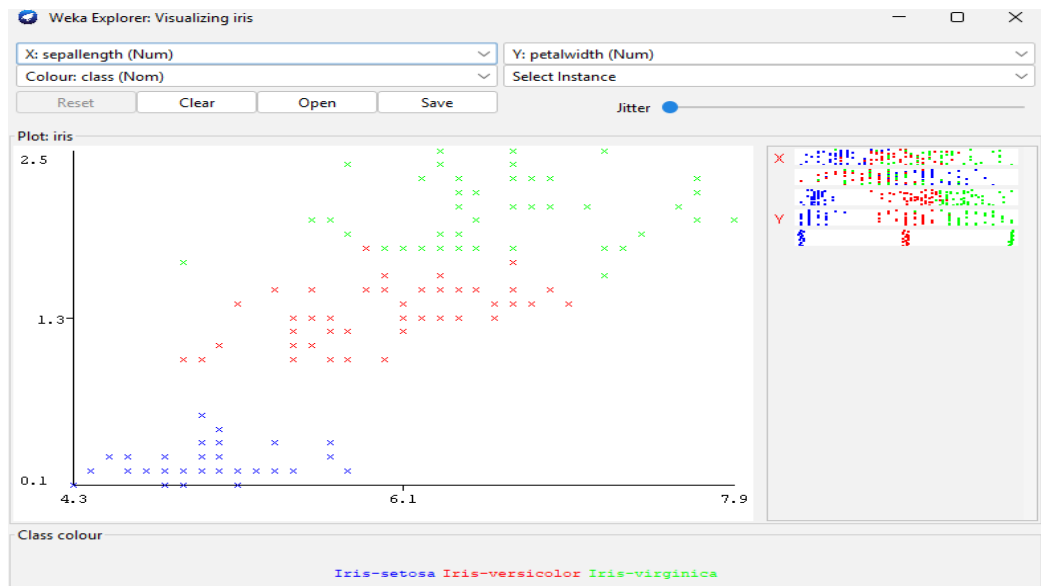
- Iris-setosa
- Iris-versicolor
- Iris-virginica



4. Click on the selected plot box to enlarge it, such as X-axis: petalength and Y-axis: petalwidth, where each class label is shown using a different color..

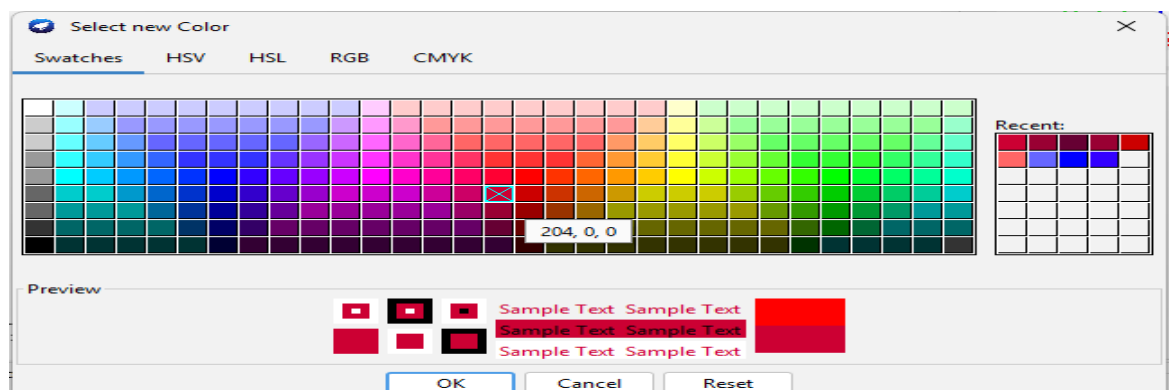
- Class label- Iris-setosa: blue color
- Class label- Iris-versicolor: red
- Class label-Iris-virginica-green

These colors can be changed. To change the color, click on the class label at the bottom, a color window will appear.



5. Changed the default colors of the three Iris classes by clicking on the color boxes at the bottom of the Visualize tab.

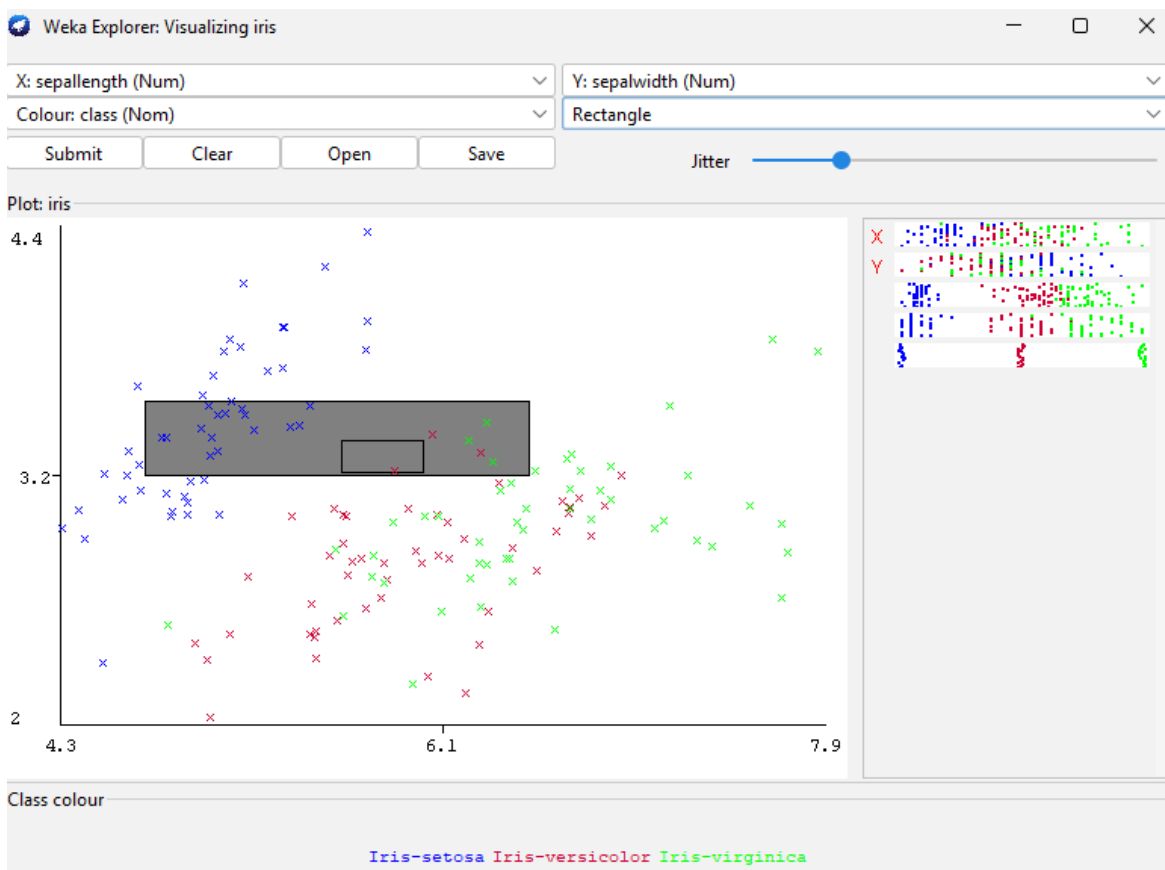
- Clicked on the color box next to each class label (Iris-setosa, Iris-versicolor, Iris-virginica).
- Selected new colors from the color picker window.
- The data points in the plot updated with the new colors.



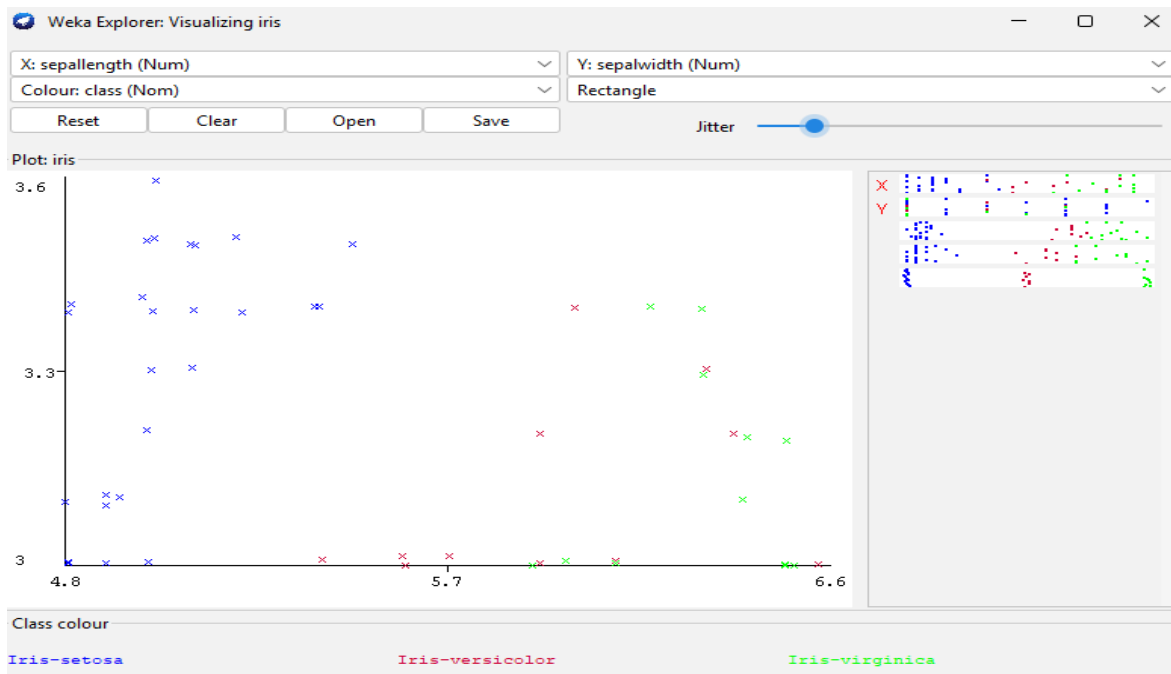
6. The Jitter slider is increased using the control at the top right of the plot. This separates overlapping data points, making them clearly visible and distinguishable. Darker areas indicate multiple instances with nearly identical values.



7. Select the Rectangle tool from the top-left side of the plot, draw a box around the desired group of data points, and click Submit to keep only the selected instances while removing the other points from the current view.



After applying the rectangle selection and clicking **Submit**, the plot displays only the selected instances, while the remaining data points are removed from the current visualization



Observation:

The Visualize tab in WEKA helps display relationships between dataset attributes through scatter plots. The Petal Length and Petal Width plot clearly separates the three Iris classes, which makes classification easier to understand. Jitter is useful for reducing overlap between points and improving visibility. The Rectangle selection tool helps filter and focus on selected data subsets. Overall, WEKA visualization is useful for analyzing data patterns, class distribution, and attribute relationships.